DriftConditions Implementation Plan

Spring 2026 Branding Abstraction Upgrade

1. Purpose

This plan covers the extraction of site identity, copy, and visual branding into a reusable abstraction layer, and the implementation of a full transactional email system. The goal is twofold: establish a complete email communication system for DriftConditions, and structure the codebase so that the platform can be forked, rebranded, and adapted to any similar community audio streaming purpose with minimal friction.

2. Goals and Scope

2.1. Goals

This subsection outlines the primary objectives of the branding and communication upgrade. Defining clear goals ensures that the implementation remains focused on providing tangible value to both the system administrators and the community contributors. The following list identifies the key areas of functional improvement and structural abstraction.

1. **Implement transactional email** — welcome, password reset, contributor digest, moderation notices, role change notifications
2. **Abstract brand identity** — site name, colors, tagline, contact info, and logo into a single config layer shared across frontend and backend
3. **Abstract copy** — surface-level text (headers, footers, onboarding, email body) separated from structure so it can be swapped per instance
4. **Enable portability** — a new deployment should require only editing `brand.js` and copy files to produce a fully re-skinned instance

2.2. Why these goals matter

DriftConditions currently has brand identity scattered across hardcoded JSX strings, CSS variables, a Tailwind config, and inline email templates in route handlers. This makes it fragile to rebrand and impossible to adapt. Pulling these into a coherent abstraction layer pays off immediately (easier to maintain) and long-term (the platform becomes a reusable foundation).

Email communication is a gap — contributors currently receive no feedback when their audio is approved, rejected, or played. Closing this loop improves contributor retention and trust.

2.3. Scope

2.3.1. Brand Abstraction Layer

- New `config/brand.js` — single source of truth for site identity
- Frontend `brand.js` that reads from or mirrors the shared config
- CSS variable generation from brand config at build time
- `mailer.js` FROM address and signature driven by brand config

2.3.2. Email Templates

- Base layout (HTML + plain text) with header/footer driven by brand config
- Templates: welcome, password reset, contributor digest, moderation notice, role change
- Template engine: Handlebars (lightweight, logic-free, widely understood)
- All templates in `AdminServer/templates/email/`

2.3.3. Copy Abstraction

- `config/copy.js` — keyed strings for email subjects, body text, frontend labels
- Frontend copy pulled from copy config rather than hardcoded JSX
- Scope limited to high-value surfaces: email body, site header/footer, onboarding

2.3.4. Out of Scope

- Full i18n / multi-language support
- Admin UI for editing copy or brand (file-based only for now)
- Marketing or promotional email flows

3. Brand Configuration

The first implementation goal establishes a single source of truth for all site identity values — name, descriptions, URLs, email addresses, and colors. Prior to this work, brand values were scattered across `index.html`, `Navigation.js`, `Homepage.js`, `mailer.js`, and `index.css` with no central home and no connection between them.

3.1. config/brand.js

`config/brand.js` is the canonical brand record for the project. It holds site name, tagline, short/long/full descriptions, site URL, OG share image, and the three email addresses used by outgoing mail (noreply, welcome, contact). It uses CommonJS (`module.exports`) for compatibility with the Node backend.

This file is symlinked into `AdminServer/node_modules/config/` via the existing config symlink, so backend code reaches it with `require('config/brand')` — no new infrastructure.

Dynamic identity values (generated project name variants, location, homepage flavor text) are intentionally excluded. Those live in `randomUtils.js` and are part of what makes this project itself.

3.2. Frontend Symlink

CRA restricts imports to within `src/`. Rather than copying the file (two sources, guaranteed to drift) or generating it at build time, `AdminClient/src/brand/brand.js` is a symlink to `../../../config/brand.js`. There is one file; both sides see it immediately on edit.

The symlink recreate command (with both ends labeled) is documented in the header of `brand.js` itself, since symlinks break silently on some git clones, rsync runs, and backup restores.

3.3. brand.css

Brand colors live in `AdminClient/src/brand/brand.css`, not in `brand.js` and not in `index.css`. This keeps CSS-native concerns in CSS where they belong, avoids the flash-of-unstyled-content that runtime JS variable injection produces, and prevents the confusion of editing CSS variables that appear to do nothing because they get overwritten at runtime.

`brand.css` is imported before `index.css` in `index.js`. `index.css` retains a comment pointing to `brand.css` for anyone who goes looking.

3.4. Mailer FROM Addresses

`AdminServer/utils/mailer.js` previously had a single hardcoded `FROM_ADDRESS`. It now imports `config/brand` and exposes three formatted From headers — `FROM.noreply` (automated system messages), `FROM.welcome` (personal welcome from Wes), `FROM.contact` (general enquiries). `sendMail()` accepts an optional `from` parameter

defaulting to `FROM.noreply`. Both `sendMail` and `FROM` are exported for use by route handlers.

3.5. Frontend Component Wiring

`Navigation.js` and `Homepage.js` previously declared `const projectName = 'DriftConditions'` locally. Both now import `brand` and read `brand.name`. `Homepage.js` also reads `brand.email.contact` to replace two hardcoded `info@driftconditions.org` mailto links.

3.6. Title and Meta Tags (deferred — Final Polish)

`index.html` title, meta description, and OG/Twitter tags remain hardcoded. The correct solution is `react-helmet-async`, which allows any component to manage `<head>` content with full access to JS values including brand config. This is slated for the Final Polish section rather than tackled here, as it requires installing a new dependency and restructuring how the app's `<head>` is managed.

4. Email System

DriftConditions currently sends one email: a password reset, assembled as inline HTML inside `authRoutes.js`. There is no welcome email, no contributor notification, no moderation feedback loop. This section establishes a proper email system — templated, brand-consistent, and wired into the routes that need it.

Please see sample emails in Appendix A.

4.1. Template Engine

Handlebars (`^4.7.9`) installed. `sendTemplate(templateName, variables, opts)` added to `mailer.js`. It reads `subject.txt`, `body.html`, and `body.txt` from `AdminServer/templates/email/<templateName>/`, compiles the inner body, wraps it in the shared layout, and calls `sendMail()`. Brand globals (`siteName`, `siteUrl`) are merged into every template context automatically. Plain text templates compiled with `noEscape: true` to prevent HTML entity encoding of apostrophes.

4.2. Layout and Partials

layout.html and `layout.txt` provide the wrapper — DriftConditions logotype (Rubik Distressed via Google Fonts `@import`), cornflower header, body slot (`{{{body}}}`), footer with site URL. Individual templates provide content only. Partials registered at startup from

`templates/email/partials/`: `contributor-tips-html.html` and `contributor-tips-txt.txt` (separate files to avoid the last-registered-wins collision between `.html` and `.txt` files sharing a name).

4.3. Templates

4.3.1. Welcome

Auto-sent on signup from `FROM.welcome`. Variables: `firstname`, `username`, `contactEmail`. Written in Wes's voice. "Drop me a line" link pre-populates subject: `{{username}} wants to be a contributor to DriftConditions`. Contributor profile link included.

4.3.2. Password Reset

Auto-sent on forgot-password from `FROM.noreply`. Extracted from inline HTML in `authRoutes.js`. Variables: `name`, `resetUrl`.

4.3.3. Role Change: Contributor

Admin-triggered from `FROM.welcome`. Variables: `firstname`, `username`. Includes contributor tips and profile page link. Sent when `notifyUser` is true on promotion.

4.3.4. Role Change: Editor

Admin-triggered from `FROM.welcome`. Variables: `firstname`. Lists new capabilities. Includes contributor tips as P.S.

4.3.5. Role Change: Mod

Admin-triggered from `FROM.welcome`. Variables: `firstname`. Describes mod responsibilities. Includes contributor tips as P.S.

4.3.6. Audio Moderation Notice

Mod-triggered from `FROM.noreply`. Variables: `firstname`, `username`, `clipTitle`, `action`, `approved` (boolean for Handlebars conditionals), `notes`. Includes profile page link. Conditional block shows "Notes on your submission" only when notes are present.

4.3.7. Contributor Digest

Stub. Deferred pending clip usage tracking (section 3.3).

4.4. UI Changes

- **User List quick edit** — "Notify user" checkbox inline with Status field (default checked, disabled until role changes; tracks `_originalRoleName` to detect change).
- **ProfileEdit** — "Notify user" checkbox inline with Role select (same disabled-until-changed behavior; `originalRoleName` stored on profile load).
- **AudioEdit** — "Notify contributor" checkbox + `moderationNotes` textarea shown conditionally when status is Approved or Disapproved (default checked).

4.5. Admin News

Admins and mods can post plain-text news items that are injected into outgoing contributor digests. It provides a lightweight broadcast channel — no per-user targeting, no threading, no replies. News accumulates until the monthly digest run clears it.

4.5.1. Data model

Items stored in `userComms` with `commType = 'admin_news'`. `userID` is the submitter's ID (not NULL — attribution matters). `payload` is `{ content, username }` — username is embedded at write time so reads require no join. `sentAt` is NULL until the monthly clear sets it to `NOW()`.

4.5.2. Backend

`adminRoutes.js` registers two routes under `/api/admin`:

- `POST /news/list` — returns all unsent `admin_news` rows, newest first. Extracts `content`, `createdAt`, and `username` from payload.
- `POST /news/create` — inserts a new row with the submitter's `userID` and `{ content, username }` payload. Validates that content is non-empty.

Both routes require the `adminNews` permission, verified via `jwt.verify` on the cookie token — same pattern as `roleRoutes.js`. Route mounted in `server.js` at `/api/admin`.

4.5.3. Permissions

New `adminNews` permission added to the `admin` and `mod` roles directly in the DB via `JSON_ARRAY_APPEND`. `editDate` bumped on both roles to force a token re-issue on the next `/check` call, so the new permission is picked up without requiring a logout.

4.5.4. Frontend

- `adminSlice.js` — `adminNewsList` and `adminNewsCreate` async thunks, registered in `store.js`.
- `AdminNews.js` — styled as an edit page (`edit-wrapper`). Queued items appear first, in digest order, as a bulleted list with small `(date by username)` attribution using `.news-contrib`. Compose textarea below. Submit disabled when field is empty. A `form-note` warns that items cannot be deleted except via the database directly.
- Navigation — "Post Updates" added under the Admin section, visible only to users with `adminNews` permission.

4.5.5. Digest integration

`getAdminNews()` in `digestRunner.js` fetches all unsent `admin_news` rows. `clearAdminNews()` sets `sentAt = NOW()` on all of them. Both `buildDigestVars` and `buildReminderVars` call `getAdminNews()` and inject `adminNews` (array of `{ content }`) and `hasAdminNews` into the template context. `clearAdminNews()` is called at the end of `runDigest()` on monthly run days (`isNthWeekdayOfMonth()`).

Daily and weekly users may see the same news items across multiple sends before the monthly clear — this is intentional. The news is worth repeating.

4.5.6. Templates

- `contributor-digest` — "Recent Updates" section inserted between "Approved — Now in the Stream!" and "Your Stats". Each item renders as its own paragraph (HTML) or block (plain text).
- `contributor-digest-reminder` — "Recent Updates" inserted between the invitation copy and the upload link.
- `user-reminder` — excluded; those users aren't contributors yet.

4.7. Route Changes

- `authRoutes.js /signup` — `sendTemplate('welcome', ...)` after successful insert; fire-and-forget with error logged.
- `authRoutes.js /forgot-password` — inline HTML replaced with `sendTemplate('password-reset', ...)`.

- `userRoutes.js /profile/edit` — checks `notifyUser` + `roleName` change; looks up user email/firstname; calls appropriate role-change template.
- `audioRoutes.js /audio/update` — checks `notifyContributor` + `status` is Approved/Disapproved; looks up creator email/firstname; calls `audio-moderation` template.

5. Final Polish

5.1. Site MetaData Branding

The one remaining hardcoded brand surface is `AdminClient/public/index.html` — the page title, meta description, and OG/Twitter social card tags. These can't be set from JavaScript at build time in a standard CRA setup, so they've been left as static strings since the project began. This section replaces them properly.

5.1.1. react-helmet-async

`react-helmet-async` is the standard approach for managing `<head>` content in React SPAs. It allows any component to declare title and meta tags as JSX, with full access to JS values including `brand`. The `async` variant is required over the original `react-helmet` for concurrent React compatibility.

Installation: `npm install react-helmet-async` in `AdminClient/`.

`HelmetProvider` wraps the app once in `index.js`. Individual pages or a root layout component set the tags via `<Helmet>`.

5.1.2. Implementation

A single `<Helmet>` block in `RootLayout.js` (or equivalent top-level component) sets the default title, description, and social tags from `brand`:

```javascript
import { Helmet } from 'react-helmet-async';
import brand from '../brand/brand';

<Helmet>
  <title>{brand.name}: {brand.tagline}</title>
```

```
  <meta name="description" content={brand.descriptionFull} />
  <meta property="og:title" content={brand.name} />
  <meta property="og:description" content={brand.descriptionLong}
/>
  <meta property="og:url" content={brand.siteUrl} />
  <meta property="og:image" content={brand.ogImage} />
  <meta name="twitter:card" content="summary_large_image" />
  <meta name="twitter:title" content={brand.name} />
  <meta name="twitter:description" content={brand.descriptionFull}
/>
  <meta name="twitter:image" content={brand.ogImage} />
</Helmet>
```

5.1.3. index.html Cleanup

Once Helmet is live, `index.html` reverts to a minimal shell — `<title>` and all brand meta tags removed, replaced with a comment explaining they're set at runtime. The `<meta name="description">` that CRA adds by default is also removed.

5.1.4. Per-page Titles (optional)

Individual pages can override the default title with their own `<Helmet>` block — e.g. `<title>Edit Audio – {brand.name}</title>`. Not required for launch but easy to add incrementally.

5.2. Audio Images

Every audio clip on DriftConditions can carry a cover image — extracted automatically from embedded MP3 art, sourced via Claude, or supplied directly by the contributor. These images flow through to mix stream metadata and the homepage rotating display, giving the station a visual identity that shifts with what's playing.

5.2.1. Configuration

Five new parameters are added to the shared config, grouped with other filesystem paths:

- `coverImageDir`: `BASEDIR + '/content/images/audio'` — destination for per-clip images
- `coverImageExt`: `'jpg'` — normalized output format

- `coverImageSize`: `[500, 500]` — normalized dimensions
- `coverImageAltDir`: `BASEDIR + '/content/images/alt'` — fallback images
- `coverImageAltNum`: `15` — count of available alt images

5.2.2. Database and Filesystem Structure

A `coverImage` text column is added to both the `audio` table and `mixQueue`. For audio clips, the field stores a partial filename — typically just the `audioID` (e.g., `1774`) — from which the full path is constructed at runtime as `coverImageDir + '/' + coverImage + '.' + coverImageExt`. Storing a text value rather than a computed path preserves flexibility for edge cases where the naming convention differs.

The `content/images/audio/` directory is created on both local and production filesystems. No subdirectory organization is needed; the flat `audioID`-keyed layout is sufficient at expected scale.

5.2.3. Alt Image Library

Fifteen existing homepage images from `AdminClient/public/img/homepage/` are copied to `content/images/alt/`, converted to JPEG, resized to 500×500, and renamed `coveralt-00.jpg` through `coveralt-14.jpg`. These serve as fallback cover art wherever a clip- or mix-specific image is unavailable. The count is fixed in config as `coverImageAltNum`; adding new alts requires both a file addition and a config update.

5.2.4. Backfill Script

A standalone script iterates all audio rows and attempts to assign a cover image to each unimaged clip. It runs in two phases:

Phase 1 — Embed extraction. For each clip, `ffmpeg` extracts the APIC (embedded cover art) frame from the source MP3 if present. The image is normalized to 500×500 JPEG and saved to `content/images/audio/{audioID}.jpg`. The `coverImage` field is set and `image-from-embed` is added to `internalTags`.

Phase 2 — Claude Haiku lookup. Clips that yielded no embedded art are passed to Claude Haiku. The clip title — which typically carries full artist and work attribution derived from the original filename — serves as the search query. Haiku identifies the likely source and returns a candidate image URL with a confidence score. Results above a calibrated threshold are fetched, normalized, saved, and tagged `image-from-haiku`. Results below threshold, or where no source is found, are tagged `image-not-found` and left without a cover image. The

confidence threshold is determined empirically from a calibration run against known clips before the full backfill is executed.

It is acceptable for a portion of the library to remain without cover images; the alt library covers those cases gracefully downstream.

5.2.5. Mix Construction Metadata

During mix assembly, `MixEngine` assigns a `coverImage` to each mix in `mixQueue`. It walks the playlist and takes the first audio clip with a `coverImage` set. If no playlist clip has an image, a random alt (`coveralt-{N}.jpg`) is selected.

`ffmpeg` embeds the resolved image as cover art in the mixed output, making it available to stream clients that support ICY or HLS metadata. Two additional metadata fields are set deliberately: the embedded title becomes `{recipe_title} – {first_playlist_title}` and the author field is set to `DriftConditions – driftconditions.org`.

5.2.6. Homepage Rotating Image

The homepage display currently cycles through images from `AdminClient/public/img/homepage/`. It is updated to read the `coverImage` field directly from the current mix's `mixQueue` entry — the same field set during mix construction in `5.2.5`. If the field is empty, a random alt from `content/images/alt/` is used as fallback. No additional playlist walking or image resolution logic is needed on the frontend.

5.2.7. User-Facing Image Fields

Four pages gain an image field: `AudioUpload`, `AudioBatchUpload`, `AudioEdit`, and `AudioView`. `QuickEdit` is excluded as a lightweight form. Each presents two input options: upload an image file directly, or supply a URL to an external image. Images are received or fetched server-side, normalized to 500×500 JPEG, and saved to `content/images/audio/{audioID}.jpg`, overwriting any previously extracted or sourced image. The clip's `internalTags` is updated to `image-from-user`.

5.2.8. Upload and Processing Routes

New or modified server routes handle image receipt and normalization for the user-facing fields. A shared utility function accepts either a file buffer or a remote URL, normalizes via `ffmpeg` to 500×500 JPEG, and writes to the appropriate path. This utility is called from the upload, batch upload, and edit routes, keeping normalization logic in one place.

6. Stretch Goals: Contributor Digest

The contributor digest is a periodic email that re-engages contributors by surfacing their standing and recent activity on the station. Unlike the per-approval emails already implemented, the digest is batched — a contributor who has five clips approved in one week gets one digest, not five emails. The system has three interlocking parts: an event queue that accumulates activity as it happens, a digest runner that builds and sends personalized emails on a schedule, and a user preference system that lets recipients choose their cadence or opt out.

6.1. Database Schema

Two schema changes are required.

6.1.1. userComms Table

An event queue that accumulates communication-worthy activity per user, decoupled from the digest sender. The `sentAt` column gates re-sending — rows with a non-null `sentAt` were already included in a digest and are skipped on future runs. Sentinel rows (e.g. `commType = 'digest_sent'`) are also inserted on each send to support the missed-send fallback mechanism.

```sql
CREATE TABLE userComms (
  commID     INT AUTO_INCREMENT PRIMARY KEY,
  userID     INT NOT NULL,
  commType   VARCHAR(32) NOT NULL,
  payload    JSON,
  createdAt  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  sentAt     TIMESTAMP NULL DEFAULT NULL,
  FOREIGN KEY (userID) REFERENCES users(userID) ON DELETE CASCADE
);
```

`commType` values: `audio_approved`, `audio_disapproved` (event rows); `digest_sent`, `reminder_sent`, `user_reminder_sent` (sentinel rows inserted after each send).

6.1.2. users Table Addition

```sql
ALTER TABLE users ADD COLUMN digestFrequency VARCHAR(16) DEFAULT
'yearly';
```

Values: `nodigest`, `daily`, `weekly`, `monthly`, `yearly`. `nodigest` opts the user out entirely, routing to immediate individual emails instead. Events continue to accumulate in `userComms` regardless of preference, in case it is re-enabled.

Role-based defaults are applied at signup and on role change (only if the user hasn't already customized their preference):

Role	Default
user	yearly
contributor	monthly
editor / mod / admin	weekly

Existing users are migrated with:

```sql
UPDATE users SET digestFrequency = 'yearly'  WHERE roleName =
'user'                     AND digestFrequency = 'daily';

UPDATE users SET digestFrequency = 'monthly' WHERE roleName =
'contributor'              AND digestFrequency = 'daily';

UPDATE users SET digestFrequency = 'weekly'  WHERE roleName IN
('editor','mod','admin') AND digestFrequency = 'daily';

ALTER TABLE users MODIFY COLUMN digestFrequency VARCHAR(16) DEFAULT
'yearly';
```

6.2. Event Queuing

Event rows are inserted in `audioRoutes.js /audio/update` as a fire-and-forget side effect of moderation actions, never blocking the response. The `notifyContributor` checkbox gates whether any notification fires at all; `digestFrequency` then determines the delivery path:

- nodigest → send an immediate `audio-moderation` email
- any other value → insert a row into `userComms` for batch delivery

Both approvals and disapprovals are queued:

```JSON
{ commType: 'audio_approved',    payload: { audioID, title, notes:
'' } }

{ commType: 'audio_disapproved', payload: { audioID, title, notes:
record.comments || '' } }
```

Notes are only populated for disapprovals. Approval emails omit notes to prevent track metadata (e.g. "from freemusicarchive.org") from appearing as editor feedback.

6.3. Digest Runner

`AdminServer/utils/digestRunner.js` implements a schedule-table architecture. A static array of schedule entries drives the entire runner — adding a new email type means adding one entry, not touching the main loop.

Each entry defines:

- `name` — identifier for logging
- `template` — Handlebars template directory name
- `commType` — sentinel value inserted into `userComms` after send
- `windowDays` — lookback window for missed-send detection (`null` = no fallback, fires every run)
- `isScheduledToday(user)` — returns true if today is the right send day; receives the user object for per-user checks (e.g. anniversary)
- `getRecipients()` — async query returning the candidate user list
- `buildVars(user)` — async function returning `{ vars, commIDs }`

The main loop for each schedule:

1. Get recipients
2. For each user: check `isScheduledToday(user)` (happy path) OR `!hasGottenLastDigest(windowDays)` (missed-send fallback)
3. If either is true: build vars, send template, mark `commIDs` as `sentAt = NOW()`, insert sentinel row via `logSent()`

Cadence is configured in `config.digest`:

```javascript
digest: {
  weeklyDay: 2,              // 0=Sun … 6=Sat; 2 = Tuesday
  monthlyWeek: 1,            // nth occurrence of weeklyDay in the month
  anniversaryWindowDays: 7, // days after anniversary to catch missed yearly sends
}
```

Five schedules:

Name	Template	Cadence	Recipients
daily-digest	contributor-digest	every run	digestFrequency='daily', has submissions
weekly-digest	contributor-digest	weeklyDay	digestFrequency='weekly', has submissions
monthly-digest	contributor-digest	nth weeklyDay	digestFrequency='monthly', has submissions
contributor-reminder	contributor-digest-reminder	nth weeklyDay	role='contributor', no submissions ever
user-reminder	user-reminder	signup anniversary	role='user', signed up ≥ 30 days ago

Missed-send fallback: `hasGottenLastDigest(userID, commType, windowDays)` queries `userComms` for a sentinel row within the window. If absent, the send fires regardless of day — catching the case where the server was down on the scheduled day. Windows: 6 days (weekly), 27 days (monthly), 350 days (yearly).

6.3.1. Unsubscribe Link

Each digest includes a one-click unsubscribe link. The link carries a signed JWT (never-expiring) with `{ userID, purpose: 'unsubscribe' }`:

```shell
GET /api/user/unsubscribe?token=<jwt>
```

This route requires no session. On success it sets `digestFrequency = 'nodigest'` and returns an HTML confirmation page. The digest and reminder footers include a separate "Manage digest preferences" link pointing to the user's profile edit page.

6.4. Email Templates

All templates use the existing `layout.html` / `layout.txt` wrappers and are rendered via `sendTemplate()` in `mailer.js`.

6.4.1. contributor-digest/

Sent to any user with at least one audio submission on their digest cadence. Sections render conditionally — a user with no new events gets a stats-only email; approved/disapproved sections only appear when there is activity.

Content:

- Disapprovals first (if any) — title + link + editor notes in blockquote
- Approvals (if any) — title + link
- Stats table: member since, last contributed, total plays
- Audio counts: contributed, waiting for approval
- Recipes (if any): contributed, waiting for approval
- Top 5 played clips (if any)
- Up to 24 clips waiting for approval (if any)
- Footer: "Manage digest preferences" link + "Unsubscribe" link

6.4.2. contributor-digest-reminder/

Sent monthly to contributors (role=`contributor`) who have never submitted any audio. Wes's voice; brief evocative list of what the station wants; link to the upload page; note that the user

must be logged in first (direct URL navigation without a session currently causes an error — see TODO). Footer: manage preferences + unsubscribe.

6.4.3. user-reminder/

Sent yearly on the user's signup anniversary to users who have never requested contributor status (role=`user`). Wes's voice; brief re-evocation of the station; repeat of the contributor ask; pre-populated mailto link. Signed `— Wes`. Footer: unsubscribe from annual reminders.

6.5. Per-User Email Preferences

`ProfileEdit.js` includes a digest frequency dropdown visible on the user's own profile edit page (and to editors/mods/admins viewing any profile):

```
None
Digest emails: [ Daily ▾ ]   (Daily / Weekly / Monthly / None)

Note: How often to receive a digest of your approvals and
contribution stats.
```

The `digestFrequency` field is included in the existing profile update payload and saved via `/api/user/profile/edit` — no new route needed. Role-based defaults are applied automatically at signup and on role promotion, but only if the user hasn't already customized their preference.

6.6. Scheduled Digest via systemd Timer

The runner fires daily at 14:00 UTC (9 AM Eastern) via a systemd timer. All cadence logic is internal — the timer fires once and the runner decides what to send to whom.

`setupfiles/digest.timer` triggers `digest.service` daily. `Persistent=true` ensures a missed run (e.g. server downtime) fires on next boot rather than being skipped entirely.

`setupfiles/digest.service` is a oneshot service that runs `scripts/run-digest.js` as the `debian` user from the `AdminServer/` working directory. `NODE_PATH` is set explicitly so Node can resolve modules from `AdminServer/node_modules/` regardless of where the script file lives.

`scripts/run-digest.js` is a thin entry point: loads `.env`, requires `digestRunner.js`, calls `runDigest()`, and exits.

To install:

```shell
sudo ln -sf /path/to/driftconditions/setupfiles/digest.service
/etc/systemd/system/digest.service

sudo ln -sf /path/to/driftconditions/setupfiles/digest.timer
/etc/systemd/system/digest.timer

sudo systemctl daemon-reload

sudo systemctl enable --now digest.timer
```

To check status:

```shell
systemctl status digest.timer
```

To view logs:

```shell
journalctl -u digest.service --no-pager
```

6.7. Implementation Order

The following sequence defines the logical progression for deploying the digest system. Each step builds upon the previous one, beginning with core database structures and ending with the scheduled server tasks. This order minimizes system downtime and ensures that background processes have the necessary data models in place before they are activated.

1. Schema migrations — `userComms` table, `digestFrequency` column — local then production
2. Event queuing in `audioRoutes.js` — both approved and disapproved, with `nodigest` branching
3. `digestRunner.js` — schedule-table architecture, `hasGottenLastDigest`, `logSent`
4. Unsubscribe route in `userRoutes.js`
5. Email templates — `contributor-digest`, `contributor-digest-reminder`, `user-reminder`

6. `ProfileEdit.js` digest frequency dropdown
7. Role-based defaults at signup (`authRoutes.js`) and role change (`userRoutes.js`)
8. `config.digest` section — `weeklyDay`, `monthlyWeek`, `anniversaryWindowDays`
9. `scripts/run-digest.js` + cron setup on server

7. Appendices

7.1. Appendix A --- Sample Emails

7.1.1. Member Welcome

Hey {{firstname}},

Welcome to DriftConditions.

You've joined a strange little audio experiment: a continuously running stream built from found sound, field recordings, music, voices, atmospheres, archival fragments, and other beautiful wreckage. It drifts somewhere between late-night radio, dream logic, and the feeling that you've tuned into something you were not quite meant to hear.

I hope you'll check in from time to time for a listen. Or, like some people, leave it on day and night and let it become the weird soundtrack of your life.

And one other thing: if you ever hear this station and think, oh, I have something for this — you probably do.

If you've got a field recording from a lonely roadside, a broken answering machine tape, a church sermon from a thrift-store cassette, distant machinery, shortwave ghosts, archival oddities, a rainstorm, a room tone, a fragment of song, an overheard voice, a strange little sound effect, or some other piece of audio debris that feels like it belongs in this world, you can become a contributor pretty easily.

Just drop me a line and tell me your username and that you want to contribute. I'll upgrade your account, and then you'll be able to upload audio for possible use in the stream.

After that, I'll send along some lightweight guidelines and what kinds of audio tend to work especially well.

So that's the shape of it: listen, lurk, drift off, come back later, or decide one day that you want to feed the machine.

Glad you found your way here.

7.1.2. Response to Request for Contribution

Hi {{firstname}},

Thanks for your interest in contributing to DrifftConditions. The station relies on user audio contributions.

Do you have audio you think would fit perfectly within the unique soundscape of the station? We welcome your contributions.

1. Sign up for an account
2. Then let us know you would like to be a contributor. We'll need to know your login.
3. We'll promote you to a contributor
4. After that, you can upload your own audio clips..

After that, you can participate in creating this ever-evolving auditory experience. Once you've signed up, reach out to us to get started. Your input helps shape the dynamic and immersive environment that makes the station so special.

So in short, as a contributor, you can submit whatever you think will add to the vibe and submit as many as you like. But I will also send you some guidelines that might help. A moderator will either approve them or offer feedback.

7.1.3. Promotion to Contributor

Hey, {{firstname}},! You've earned a promotion to contributor!

As a contributor, you can add audio that will be used to make the mix for the station.

To contribute, select Add New Audio from the menu in the top left. You will be invited to

upload your audio files.

{{Tips Reminder}}

7.1.4. Tips Reminder

Here's some tips that will help make sure your contributions are successfully added to the mix:

Title
Use simple meaningful titles that credit the original source. These titles will appear in the playlist.

Classification
Classifications are broad categories that describe the type of audio. **Pro-tip:** Use as few classifications as applicable. Here are the meanings for each classification:

- **Ambient**: Music or sounds that form an ambient bed for other audio, e.g., drone, ambient music, longer static audio, minimalist music
- **Atmospheric**: Includes many field recordings, and longer found sounds, e.g., city ambiance, cafe recordings, crowd noise
- **Environmental**: Recordings of natural sounds, e.g., distant thunder, bird calls, rainfall, ocean waves
- **Premixed**: Already assembled mixes that would not benefit from additional mixing, e.g., radio show/podcast excerpts, mixed music tracks
- **Soundscape**: Complex audio environments combining various sounds, e.g., urban soundscape, virtual environment audio, theatrical sound design
- **Archival**: Historical recordings or sounds, e.g., old radio broadcasts, historical speeches, old TV shows
- **Spoken Word**: Spoken Word recordings, e.g., talks, poetry readings, speeches, interviews
- **Narrative**: Recordings that tell a story, e.g., radio dramas, storytelling sessions, narrative podcasts
- **Instructional**: Educational or instructional audio, e.g., language lessons, how-to guides, training programs
- **Vocal Music**: Music that prominently features vocals, as opposed to instrumental
- **Instrumental**: Music without vocals, e.g., classical music, instrumental jazz, ambient instrumental tracks

- **Experimental**: Non-traditional or avant-garde audio, e.g., noise music, sound experiments, unconventional audio compositions
- **Digital**: Digital sounds and effects, e.g., synthesizer music, electronic beats, computer-generated sounds
- **Effect**: Short recordings that can be layered over other audio, e.g., sound effects, audio stingers, transition sounds
- **Other**: Miscellaneous sounds that do not fit into other categories, e.g., unique or rare audio clips, uncategorized recordings

Tags

Tags describe the topic, theme, or texture of the audio, such as 'rainy night', 'jazz', or 'whispering'. Use tags to highlight specific elements or moods in your audio. Separate tags with commas. **Pro-tip:** Use as many simple tags as you can think of.

Since tags are used to group similar audio, keep tags simple and plentiful. Rather than compound tags like:

```
1950s-educational-film, propaganda movie, or teenage-angst
```

use simple tags like:

```
1950s, educational, instructional, propaganda, teenage,
teenager, angst
```

Don't worry about providing all possible plural tags as the system will already try to work that out automatically.

A Word About Copyright

In general, we think copyright is too often a way of locking in the profits of corporations, however we do want to show respect for other artists. So far, I've used only stuff that was either:

- in the public domain, e.g., stuff on archive.org or published before 1924
- creative commons, e.g., material on freemusicarchive.org or freemusicarchive.org
- public performance captured by a spectator, e.g., street performers, gamelan beleganjur
- so old and influential, in our opinion, it could be said to be owned by everyone, e.g., Jelly Roll Morton, Texas Swing, Ink Spots

- so obscure, the crime of stealing it is offset by the opportunity that someone actually gets to hear it, e.g., Tuvan throat singers, Baka water music

Hopefully, this will be helpful (and not overwhelming) information that will guide you as you contribute.

7.1.5. Promotion to Editor

Maybe this should be a more general promotion email which lists what they are able to do. Not sure.

Hey {{firstname}},

The material you've been submitting has been spot-on. So you've earned a promotion to editor! You'll see a few more actions on the menu:

- List All Audio
- Edit Audio
- Submit Batch Audio

You'll also notice that you can self-approve your submissions, even when you submit them.

Use your new powers wisely and carefully!

If you have questions, don't be afraid to ask.

7.2. Appendix B: Database Schema Diff

Two schema changes support the digest system.

7.2.1. New table: `userComms`

Accumulates communication-worthy events (audio approved, audio disapproved) with a `sentAt` field that gates re-sends. Also used for sentinel rows that record digest sends, enabling the missed-send fallback logic.

```sql
CREATE TABLE userComms (
  commID    INT          NOT NULL AUTO_INCREMENT,
  userID    INT          NOT NULL,
  commType  VARCHAR(64)  NOT NULL,
  payload   JSON,
```

```
  createdAt DATETIME     DEFAULT CURRENT_TIMESTAMP,
  sentAt    DATETIME     DEFAULT NULL,
  PRIMARY KEY (commID),
  KEY idx_userID (userID),
  KEY idx_commType (commType)
);
```

`commType` values in use: `audio_approved`, `audio_disapproved` (event rows); `digest_sent`, `reminder_sent`, `user_reminder_sent` (sentinel rows).

7.2.2. New column: `users.digestFrequency`

Controls how often each user receives a digest. Defaults to `'yearly'`; users can change it on their profile edit page.

```SQL
ALTER TABLE users ADD COLUMN digestFrequency VARCHAR(16) DEFAULT
'yearly';
-- Values: 'nodigest', 'daily', 'weekly', 'monthly', 'yearly'
```

Role-based defaults applied at migration time:

```SQL
UPDATE users SET digestFrequency = 'yearly'  WHERE roleName =
'user';

UPDATE users SET digestFrequency = 'monthly' WHERE roleName =
'contributor';

UPDATE users SET digestFrequency = 'weekly'  WHERE roleName =
'editor';

UPDATE users SET digestFrequency = 'daily'   WHERE roleName IN
('mod', 'admin');
```

```sql
ALTER TABLE users MODIFY COLUMN digestFrequency VARCHAR(16) DEFAULT
'yearly';

ALTER TABLE users MODIFY COLUMN digestFrequency VARCHAR(16) DEFAULT
'yearly';
```

On role promotion, `userRoutes.js` automatically updates `digestFrequency` to the new role's default — but only if the user hasn't customized it away from their previous role's default.

7.3. Appendix C: Server Changes

7.3.1. New files

- `AdminServer/utils/digestRunner.js` — schedule-table digest runner. Five schedules (daily, weekly, monthly, contributor-reminder, user-reminder), each defining cadence, recipient query, var builder, commType sentinel, and missed-send window. Main loop is ~20 lines; all per-schedule logic is data, not code.
- `AdminServer/templates/email/contributor-digest/` — HTML and plain-text digest templates. Sections: approvals, disapprovals (with editor notes), stats (member since, audio counts, plays, pending clips, recipes, top plays).
- `AdminServer/templates/email/contributor-digest-reminder/` — HTML and plain-text reminder for contributors who have never submitted audio. Friendly nudge with upload link and unsubscribe/prefs footer.
- `AdminServer/templates/email/user-reminder/` — HTML and plain-text anniversary nudge for users who have not become contributors. Sent once per year within a 7-day anniversary window.
- `scripts/run-digest.js` — thin entry point. Loads `.env`, calls `runDigest()`, exits. Designed to run from `AdminServer/` as working directory.
- `setupfiles/digest.service` — systemd oneshot service. Runs `run-digest.js` as `debian` user; sets `NODE_PATH` explicitly so Node resolves modules from `AdminServer/node_modules/`.
- `setupfiles/digest.timer` — systemd timer. Fires daily at 14:00 UTC (9 AM Eastern). `Persistent=true` catches missed runs on server restart.

7.3.2. Modified files

- `AdminServer/routes/userRoutes.js` — role-based `digestFrequency` defaults on promotion; `GET /api/user/unsubscribe` route verifies signed JWT and sets `digestFrequency = 'nodigest'`.

- `AdminServer/routes/audioRoutes.js` — inserts `audio_approved` / `audio_disapproved` events into `userComms` on moderation status change, independent of the notify checkbox.
- `AdminClient/src/pages/ProfileEdit.js` — digest frequency dropdown; added `yearly` option.
- `AdminClient/src/utils/authUtils.js` — auth redirect chain: unauthenticated users redirected to `/signin?next=<url>`; after login, forwarded to original URL.
- `AdminClient/src/pages/Signin.js` — reads `?next=` param after login and navigates there.
- `AdminClient/src/store/authSlice.js` — `initialState.user = { permissions: [] }` to prevent crash on cold load.
- `AdminClient/src/layouts/RootLayout.js` — render guard holds protected pages until auth check completes and redirect fires.

7.4. Appendix D: Future Enhancements and Technical Debt

7.4.1. Known gaps

- **OAuth signin loses `?next=` redirect.** When an unauthenticated user follows a protected link and chooses OAuth login, the destination URL is lost during the OAuth round-trip. Fix requires threading `next` through the OAuth `state` parameter on both initiation and callback. Tracked in `TODO.md`.
- **Digest not yet wired to `clipUsage`.** The contributor digest template has a placeholder for "clips used in recipes" but `clipUsage` tracking (section 3.1) is not yet implemented. The digest launches without it; the section can be added once tracking is live.
- **Audit table still empty.** The `audit` table exists in the schema but no code writes to it. Auth and user management events are not yet logged there.

7.4.2. Technical debt

- `digest.service` **runs in Ethereal mode** (`NODE_ENV=development`) pending verification of the first real send. Remove this line and redeploy once email output is confirmed correct in preview.
- `userComms` **sentinel rows from testing** may delay first real sends for some users by up to the cadence window (6 days for weekly, 27 days for monthly). Clear sentinel rows before going live if immediate sends are desired.
- `NODE_PATH` **workaround in `digest.service`.** The service sets `NODE_PATH` explicitly because Node resolves modules relative to the script file's directory (`scripts/`), not the working directory (`AdminServer/`). A cleaner long-term fix would be a root-level `package.json` or moving `run-digest.js` into `AdminServer/`.

7.5. Appendix E — Document Style Guide

This section details the standardized formatting applied to the document.

7.5.1. Headers and Section Titles

This style dictates the formatting for all section titles and headings (Heading 1, Heading 2, Heading 3, etc.) to ensure clear hierarchy and visual distinction from body text. All headers must be formatted exclusively using these standard Google Docs heading styles rather than custom text styles.

- **Font Family:** Arial
- **Font Size:** Varied (e.g., 20pt for Heading 1, 16pt for Heading 2, 14pt for Heading 3)
- **Font Weight:** Bold
- **Application:** Applied exclusively to section titles and subtitles, using the designated Docs Heading styles.

Structural Patterns: H1 headings (e.g., "1. Goals and Scope") must use the "Heading 1" doc style; H2 headings (e.g., "1.1. Goals") must use the "Heading 2" doc style; and this hierarchical pattern must be followed for all subsequent heading levels.

7.5.2. Body Paragraphs and Standard Prose

This section defines the baseline formatting for all narrative and descriptive text in the document. Standardized prose ensures optimal readability and consistency across all sections, distinguishing it clearly from headings and technical lists.

- **Font Family:** Arial
- **Font Size:** 11pt
- **Line Spacing:** 1.15
- **Spacing (Before Paragraph):** 0pt
- **Spacing (After Paragraph):** 12pt
- **Application:** Applied exclusively to normal body text, excluding headings, titles, and list items.

7.5.3. Inline Technical References

This style is used for all short code references embedded within the main text, such as variable names, file paths, API routes, and terminal commands. It serves to visually isolate code elements from surrounding narrative text.

- **Font Family:** Roboto Mono (Monospaced)
- **Text Color:** Gemini Green (#188038)

- **Background:** None
- **Use Cases:** Inline code snippets (e.g., `clipUsage` table), variable names (e.g., `audioID`), file paths (e.g., `config/config.js`), and console output text.

8.4.4. Code Block Syntax Highlighting Style: One Light (Notion variant)

7.5.4. Code Block Syntax Highlighting Style: One Light (Notion variant)

Background: Light gray (`#f7f7f7` or similar off-white) **Default text:** Dark gray (identifiers, punctuation, table/column names)

Token colors:

- **Keywords** (`ALTER`, `CREATE`, `ADD`, `NOT`, `NULL`, `ON`, `DELETE`): Medium blue (`#0070c1` or similar)
- **Types** (`INT`, `VARCHAR`, `DATETIME`): Bright blue (`#0099cc` or similar)
- **Numeric literals** (`255`, `32`, `191`): Orange-red (`#e06c00` or similar)
- **Comments** (`-- ...`): Muted pink/mauve (`#c678a0` or similar)
- **Language label** (`SQL`): Small-caps, light gray, non-highlighted

Typography: Monospace, medium weight, line height of 1.15. Token colors are distinct but not garish — optimized for readability over visual drama.

```sql
-- Existing 'users' table update (only adding utility columns)
ALTER TABLE users
  ADD COLUMN displayName VARCHAR(255),
  ADD COLUMN lastLoginAt DATETIME;

-- New 'userIdentities' table to support multiple linked OAuth accounts
CREATE TABLE userIdentities (
  userId INT NOT NULL, -- Foreign key to the users table
  oauthProvider VARCHAR(32) NOT NULL,
  oauthId VARCHAR(191) NOT NULL,
  PRIMARY KEY (oauthProvider, oauthId),
  FOREIGN KEY (userId) REFERENCES users(userID) ON DELETE CASCADE
);

-- Allow null names since OAuth providers may not supply them
ALTER TABLE users
```